

CONSTRAINT-BASED PLAYLIST GENERATION USING AUDIO FEATURE SIMILARITY 2025

Thomas Pietrovito

University of Victoria

Computer Science and Engineering

thomhplatt@uvic.com

Noah Hicks

University of Victoria

Computer Science and Engineering

noahjhicks@uvic.ca

ABSTRACT

In this paper we propose the creation of a constraint-based playlist generator utilizing both features extracted from music through Music Information Retrieval (MIR) techniques and associated music metadata for playlist candidate retrieval. The approach taken in this proposal aims to augment a music recommender through the addition of user specified constraints, enabling the recommendation procedure to be further personalized to suit the users preferences. Following preliminary research, we propose a method that employs a k-nearest neighbors approach utilizing features selected through user specification for comparison. We argue that this approach enables more personalized results for playlist generation and potentially lays the groundwork towards Natural Language Processing (NLP) generated playlists.

1. INTRODUCTION

Playing a key role in the field of MIR research, music recommenders have been used to great effect to facilitate the discovery of new music for users across various platforms. An example being Spotify's "radio" feature, which allows users to automatically create playlists based on similarity to a selected song, album, or artist. While these can assist users in potentially discovering new music, the methods employed for guiding recommendations remain limited, with a users only input being that of the selected object itself. This can lead to cases where the songs returned by recommendation may not align with a user's particular taste, as perhaps an undesired genre was returned due to structural similarities in a song, or the user wished to emphasize certain aspects of the song to curate recommendations.

To address this issue, we propose a system that employs the use of user specified constraints to further refine the recommendation process. The high-level usage of the system will consist of a user creating a structured query

in order to generate a playlist of recommended songs, for which varying fields will be specified. The first of these will be the size of the playlist to be generated, creating in turn a playlist of size 'k' songs ranked by similarity. Following this, a 'seed' song will then be specified, this will be used to provide the basis for content-based recommendation by providing a set of 'features' (i.e. valence, energy, danceability) for use of similarity measures against other songs. Next, a user will then be able to specify a set of constraints to be imposed for song selection. This will allow for songs that do not abide by the specified constraints to be filtered and excluded before direct comparison occurs (i.e. exclude all songs whose genre is jazz). Lastly, a user will also have the ability to specify which features of a song will be used for direct comparison via similarity measures. Enabling direct comparison to use only a specified subset of features to determine relative similarity. To enable this, we provide a struc-

tured querying language in JSON format, for which the following fields are present: "K", an integer specifying the number of recommended songs to return, "Seed", a string specifying the 'seed song' to be used, "Features" consisting of the features inherent to each song and its associated weight to use for euclidean comparison, and lastly "Constraints" containing "Include" and "Exclude". Through use

of the pipeline outlined, the framework is provided to enable a high-level query such as "Given Paranoid Android by Radiohead, create a playlist of 10 songs comparing only tempo and valence but not belonging to genre 'Pop'" to be represented as a structured query and subsequently retrieve similar songs meeting the desired criteria.

2. PROJECT COMPONENTS

2.1 Library

The project relies on the `Music4All` (M4A) dataset at the recommendation of George Tzanetakis. This dataset provides pre-extracted and normalized MIR data such as tempo, valence, danceability, and energy. Further, it also contains meta data features such as release date, genres, and language.



2.2 Dataset

The data from the library will be further normalized and stored in a `Pandas` dataframe alongside all relevant metadata. Further, the dataframe will contain all extracted audio features – ideally as a fixed-length numeric value.

2.3 Tools

1. `PyParsing` will be used to parse the expressions provided in the `JSON` queries
2. `Pandas` will be used to build a dataset that can be filtered with `SQL` commands.
3. `Numpy` will be used for basic vectorization and manipulation.
4. `Matplotlib` will be used for basic data visualization.
5. `JSON` files will be used to store and structure the queries.

3. TIMELINE

This project has clear milestones and meaningful deliverables. The following is a strategic and thoughtful approach to ensure adequate time for all sections and an equal workload for all members.

3.1 Importing Dataset Initializing Repositories - Due 2026-02-17

Music Maven will be using the `music4all` (M4A) dataset. The first goal is to obtain a copy of the dataset – either locally or shared. Ideally, we will obtain a copy of just the dataset labels (without audio) for local testing and training. We will also create branches from the main and ensure we are able to safely push and pull.

Milestone Progress: This milestone was met on 02/25/2026

3.2 Handling Preprocessing Basic Queries - Due 2026-02-24

Ensure the M4A dataset is adequate for a similarity search. That is, removing unneeded features, scaling numeric values and encoding categories. The pruned and processed dataset will be used for any training and querying. We can also begin to query the dataset using basic `Pandas` `Dataframe` functions.

MIR Features in M4A:

1. Dancability
2. Energy
3. Key
4. Mode
5. Valence

Metadata Features in M4A

1. ID
2. Album Name
3. Release Date
4. Genres

Milestone Progress: This milestone was met on 02/25/2026

3.3 Creating Query Structure - Due 2026-03-03

Define a query language that allows the user to set constraints. This will be done using a `JSON` query, as it will be the most extensible for NLP and linking to an LLM in the future.

JSON Query:

```
query = {
  "k": 10,
  "seed": {
    "artist": "Gorillaz",
    "song": "Feel Good Inc.",
    "album_name": "",
  },
  "features": {
    "popularity": 0,
    "danceability": 1.0,
    "energy": 1.0,
    "key": 0,
    "valence": 1.0,
    "tempo": 0,
    "duration_ms": 0,
  },
  "constraints": {
    "include": {
      "genres": [],
      "lang": ["'en'"],
      "artist": [],
      "song": [],
      "album_name": [],
      "release": [">= 2010 & <= 2020"]}
    "exclude": {
      "genres": ["pop"],
      "lang": [],
      "artist": ["'Gorillaz'"],
      "song": [],
      "album_name": [],
      "release": []},
  },
}
```

Milestone Progress: This milestone was met on 03/12/2026

3.4 K-Nearest Songs - Due 2026-03-10

To return the k-nearest songs that match the constraints defined in the query language, we must create a filtering system that prunes the dataset before the similarity search. By eliminating unneeded tuples from the dataset, we will improve the running time of our system and also be able to handle hard constraints (i.e, release date ≥ 2014) with ease. Some of the potential distance metrics to be used are as follows:

Weighted Euclidean:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^N w_i (x_i - y_i)^2} \quad (1)$$

Cosine Distance:

$$d(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \quad (2)$$

Mahalanobis Distance:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^T \Sigma^{-1} (\mathbf{x} - \mathbf{y})} \quad (3)$$

Minkowski Distance:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}} \quad (4)$$

Canberra Distance:

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n \frac{|x_i - y_i|}{|x_i| + |y_i|} \quad (5)$$

Chebysev Distance:

$$d(\mathbf{x}, \mathbf{y}) = \max_i (|x_i - y_i|) \quad (6)$$

Bray-Curtis Distance:

$$d(\mathbf{x}, \mathbf{y}) = \frac{\sum_{i=1}^n |x_i - y_i|}{\sum_{i=1}^n |x_i + y_i|} \quad (7)$$

Milestone Progress: This milestone was met on 03/15/2026

3.5 Feature Extraction - Future Work

This stage of the project will focus on extracting meaningful audio features from the processed PCM signals. The goal is to derive numerical representations of the qualities that define the songs in the dataset. That is, our aim is to extract features such as tempo (BPM), average RMS, rhythmic and timbre indicators and store them as numeric values alongside the categorical features in the Pandas dataframe. For features computed over time, the mean and standard deviation will be used to calculate a fixed-length value that can be used in similarity-search algorithms.

To facilitate this, .mp3 files will be preprocessed in a pipeline that does the following:

1. Decode .mp3 files into raw PCM data via `librosa`.
2. Resample PCMs at 22.05 kHz to reduce computational cost while preserving relevant information for feature extraction.
3. Convert audio signals to mono to eliminate channel variation.
4. Normalize the audio amplitude to ensure consistent feature scaling across tracks.
5. Store processed PCMs with unique song ID's (SID) and all metadata for next steps.

3.6 Handling Natural Language Queries - Future Work

Finally, should time permit, we aim to extend our system to handle queries inferred from natural language input. This will allow for queries such as "Create a playlist of 100 songs similar to the Beatles and Kinks, within the year range of 1970-1990" to be tokenized and represented in our structured query language.

4. PROGRESS REPORTS

Thomas Pietrovito:

PL.1 (Basic) — Load M4A feature and metadata into a structured pandas DataFrame

This task was achieved by saving all metadata CSVs into the working directory and loading them into a notebook. Since there are several CSVs containing different metadata, we needed to merge them in such a way that we could query and parse the data as a single dataframe. To accomplish this, we concatenated the metadata and joined on the unique song ID.

PL.2 (Basic) — Normalize data and preprocess dataset for searching

The data in the M4A dataset is normalized to itself. That is, some features such as 'Energy' and 'Danceability' are normalized between 0-1, while 'Popularity' is normalized between 0-100. Normalizing the data with a logarithmic scale or min-max approach is among the next steps,

however, we were able to still produce meaningful results given the data in its current normalized form.

PI.3 (Expected) — *Implement a search algorithm that returns a desired number of closest matches*

This milestone was accomplished through the implementation of a k-nearest neighbors search function that computes the Euclidean distance between a query feature vector and all songs in the dataset. The algorithm ranks results based on similarity and returns the top K closest matches.

Ultimately, the proposed similarity search chosen was a weighted Euclidean search, which measures the distance between vectors. The choice of this search metric was solidified following comparison against varying metrics evaluated through cohesion of the clusters produced, with the metrics listed in section 3.4. However, it should be noted that these comparisons were conducted within a euclidean space, potentially leading to quality of certain metrics such as "cosine" being determined in a flawed manner. Potential future work will be done to provide a means of more comprehensively comparing distance metrics through quantifiable means, rather than relying on visual observation. With a potential methodology being something akin to the Kendall Tau method, or some other form of metric averaging across results with an established ground truth (using attributes such as shared genres, tags, etc) Some of the results of candidate metrics are as shown below:

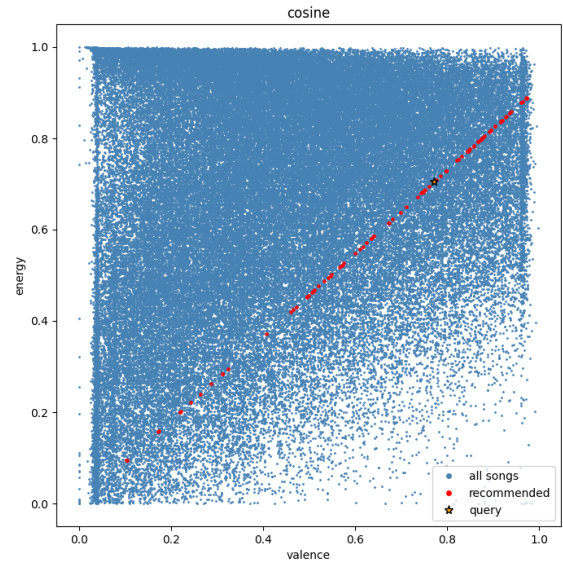


Figure 2. Cosine Clustering for Valence vs Energy

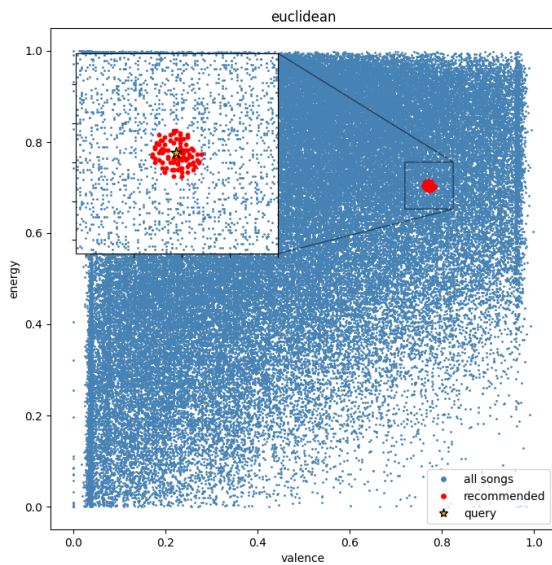


Figure 1. Euclidean Clustering for Valence vs Energy

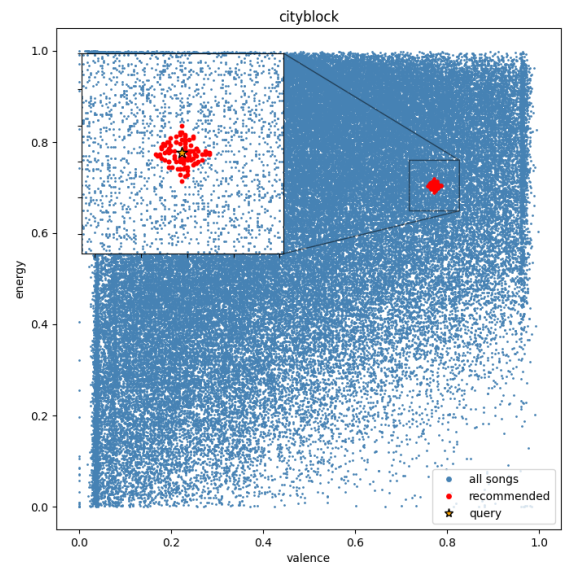


Figure 3. Cityblock Clustering for Valence vs Energy

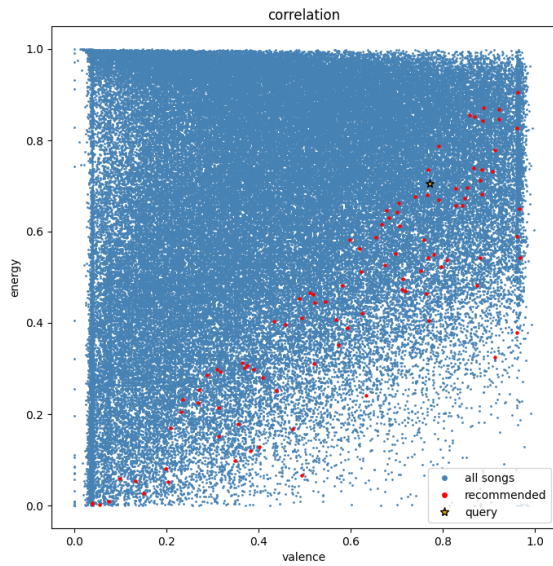


Figure 4. Correlation Clustering for Valence vs Energy

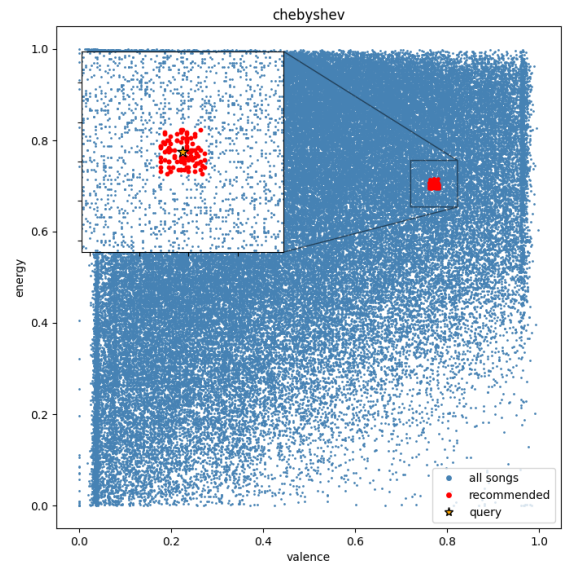


Figure 6. Chebyshev Clustering for Valence vs Energy

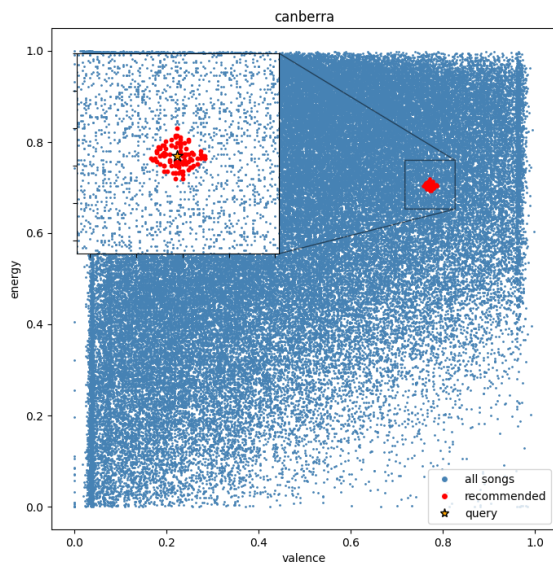


Figure 5. Canberra Clustering for Valence vs Energy

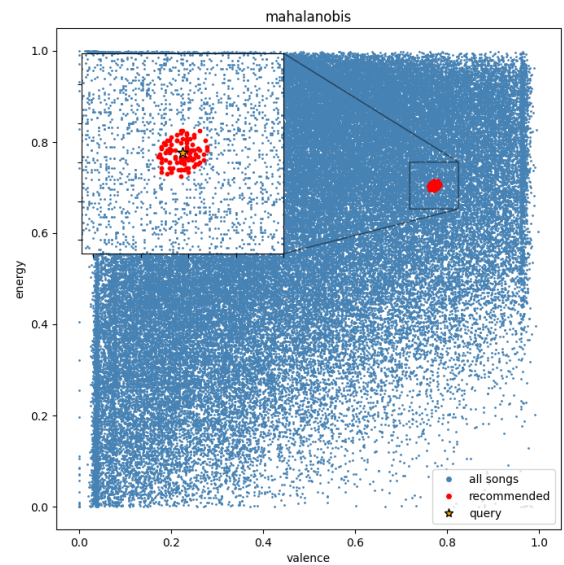


Figure 7. Mahalanobis Clustering for Valence vs Energy

PI.4 (Expected) — *Extend search algorithm to accept weights and feature importance*

This milestone was achieved by extending the Euclidean distance search to incorporate feature weights. By scaling feature differences according to user-defined importance values, the system allows certain musical attributes to influence similarity more strongly, enabling more personalized playlist generation.

PI.5 (Advanced) — *Prototype an interface that allows*

features and weights to be offered from natural language input

The next steps of the project will involve extending the input to use NLP that will generate a formatted query file. By using LLMs and tokenizing input into meaningful lemmas, we aim to operate the KNN search through natural language.

Lessons Learned

One of the most valuable lessons I learned this project was the importance of NumPy and vectorization. Further, the basic manipulation and management of large datasets in a basic SQL fashion was valuable information. Further, while researching distance metrics I learned about several that I had not yet encountered in class. For example, the Mahalanobis Distance metric, which takes in to account the standard deviation of high-dimension vectors and thus is a valuable tool for outlier detection. Or the Canberra distance, which is effectively a weighted Manhattan distance and also a proficient outlier detection tool. Something else that this project taught me was the challenges involved with working with an unlabeled dataset. That is, during the testing of the different distance measurements we were faced with the constraint of not having a ground-truth to measure against. This made it very difficult to establish meaningful accuracy metrics as we lacked "Dummy" classifiers. Further, we also realized the role of objectivity and subjectivity in our project. That is, we were able to produce numerically similar songs (matching near identical metrics in terms of valence, energy, etc.) but realized that numeric similarity does not equate to subjective preference. There is potential to prune lesser-known candidate songs by increasing the constraints on popularity, but there is still no guarantee that the user will be given songs that they actually enjoy. This leads to some potential future work, which may take into account the users previous queries, other playlist additions, most frequently listened to songs etc. The one lesson I wish I learned in this project was actually extracting and normalizing the audio data with techniques taught in class. The M4A dataset is a valuable resource and ultimately made this project feasible, however, it did accomplish the feature extraction portion of the project with no involvement from the members.

Noah Hicks:

PI.1 (Basic): Implement reading/writing to JSON files

This milestone was reached through the implementation of a basic JSON parser. Through use of this parser, queries can be retrieved from a specified JSON file and converted into a dictionary representation, enabling queries to be represented in a structured format. Use of this format allows for inference of the attribute that the expression pertains to by reference of the dictionary key used, enabling database filtering operations to be automated via our provided functions.

PI.2 (Basic): Map JSON query to pandas dataset filtering functions

Using the information provided in a parsed JSON query, this milestone was achieved through use of functions pro-

vided by the libraries Pyparsing and Pandas to enable parsing and subsequent evaluation of querying expressions provided. With the presence of "include" and "exclude" fields provided in the query, users are able to apply boolean expressions in the form ">= 1980 & <= 2020" to a specified database attribute such as "releases" to further refine the KNN results returned. Pyparsing was utilized to inject attribute names into the expression provided through reference to the category it was placed into (via dict key). Allowing for a query containing "include" : 'releases' : [">= 1980 & <=2020"] to be converted into the expression "releases>= 1980 & releases<=2020", these expressions are then fed into the Pandas eval() function to return a mask vector corresponding to the expression provided (with masks corresponding to expressions in "exclude" being negated). This approach was chosen for a

couple of reasons. The first pertains to both ease of use and consideration of future pipelines, as the use of "buckets" for both "include" or "exclude" allows for complex filters to be decomposed into a list of simple boolean expressions for each specified column, eliminating both ambiguity in constructed expressions and potentially providing a framework for future NLP integration via tokenization and mapping to "buckets". The other reason resulted from the

way that genres are represented in the provided Music4All CSVs, as each column for "genres" contains a string of values, potentially containing multiple entries. Thus, expressions such as "Genres=='jazz'" will only return matches that equal 'jazz', with entries containing 'jazz, blues' not being matched. Thus, as this necessitates a function like .apply() to parse and search within these strings, we can then check the key of the category to determine when we are dealing with "genres" expressions and apply the needed function. Lastly, some additional work is still needed to

handle boolean expressions applied to genres, as currently the system only supports a search within the list of provided values (amounting to essentially an "OR" statement for each). Also of note, any strings attributes like artist, song, lang, etc need to have their values passed in with single quotes like "'en'", this will need to be fixed in future revisions. **(This issue has now been fixed in later revisions)**

PI.3 (Expected): Extend input to accept inferred weights (e.g., "faster", "more energetic")

Unfortunately, this milestone was not able to be fulfilled within the time frame this project permitted. This was largely due to the unforeseen challenges that semantic inference imposed, as the process of mapping a term to our defined query structure quickly became challenging when having to account for inherent ambiguity present within a statement, with some terms potentially having a clear mapping such as energetic to energy while a statement like "more lively" might require more consideration (i.e this could potentially correspond to features valence, energy, danceability, how do then assign weights in this context?). Potential future work for this could be done through a method such as Retrieval-Augmented Genera-

tion (RAG) to establish relations/mappings.

PI.4 (Expected): *Prototype an NLP pipeline (via spaCy or LLM API) that converts natural language into valid JSON query* Due to the complex nature of this task, this goal was unable to be met within the context of this project. Attempts were initially taken to implement a basic NLP pipeline through means such as spaCy and Huggingface transformers (Flan-T5), but due to the unsatisfactory results obtained any further progress was ultimately ceased. In the case of frameworks such as spaCy, trying to directly enforce semantic rules seemed that it would lead to a very constrained input range, which ultimately did not conform with the goal initially set. In the case of self-hosted NLP transformers such as Flan-T5, the results obtained from these models proved to be too inconsistent for reliable use. Conversely, models such as ChatGPT provided fairly consistent results for converting queries into the structured JSON format, however, due to the price of use it would be ideal to determine a self-hosted solution in the future.

PI.5 (Advanced): *Explore failure and ambiguity in natural queries, local tasks vs generative tasks*

In a similar fashion to milestones 3,4,5. This goal was unable to be met within the given time frame due to the relative complexity of the proposed topic. Future work to be done could potentially include a creation of 'ground truths' to provide a means of determining the relative quality of mappings inferred from a given statement. This can then be further used to refine and potentially train the future NLP model(s) used.

Lessons Learned

During the course of this project, many lessons were learned relating to the usage and manipulation of large datasets through use of frameworks such as Pandas and Numpy, and the parsing/mapping of JSON query entries to database operations. During the initial stages of implementation, research was first conducted into the creation of dataframes via Pandas to store the csv files provided by Music4All within the context of the recommender. The implementation of this allowed me to understand how a data processing pipeline can be created, with metadata spread across multiple csvs being able to be combined, pre-processed, and subsequently operated upon. Following this, an aspect that initially proved a challenge was the mapping of query features such as the imposed constraints to functional database operations. Having to account for this led me to learning the basics of the library "Pyparsing", for which semantic rules could be established to extract the operators and the operands from a query and reconstruct it into a Pandas friendly statement (i.e year : [">= 2010 & <= 2020"] mapping to "year>=2010 & year <= 2020". This further resulted in me learning how vectorized masks could be applied to a dataframe through operations such as .eval(), which allowed for efficient filtering imposed by the specified constraints. After the framework was created for loading, processing, and filtering the provided data, the next stage in the project consisted of employing distance metrics to determine the most similarly songs. Through

this process I learned how methods such as k-NN could be used with different distance metrics to scan over the filtered data and return the K top matches. This also taught me how closely related Pandas and Numpy are, with Pandas providing the ability for a data frame to be converted to a Numpy matrix, allowing for Numpy and Scipy operations to be conducted upon the data contained within (which is how we were able to implement our various distance metrics).

5. REFERENCES

- [1] J. P. Bello, "Grouping Recorded Music by Structural Similarity," in *Proc. ISMIR*, 2009.
- [2] H. Homburg et al., "A Benchmark Dataset for Audio Classification and Clustering," in *Proc. ISMIR*, 2005.
- [3] Y. Liu et al., "Clustering Music Recordings by Their Keys," in *Proc. ISMIR*, 2008.
- [4] D. Kim et al., "A Music Recommendation System with a Dynamic K-Means Clustering Algorithm," in *Proc. ICMLA*, 2007.
- [5] E. Pampalk, "An Implementation of a Simple Playlist Generator Based on Audio Similarity Measures," in *Proc. ISMIR*, 2006.
- [6] F. Maillet et al., "Steerable Playlist Generation by Learning Song Similarity," in *Proc. ISMIR*, 2009.
- [7] B. McFee and G. Lanckriet, "The Natural Language of Playlists," in *Proc. ISMIR*, 2011.
- [8] D. Bogdanov and P. Herrera, "How Much Metadata Do We Need in Music Recommendation?," in *Proc. ISMIR*, 2011.
- [9] M. F. McKinney and J. Breebaart, "Features for Audio and Music Classification," in *Proc. ISMIR*, 2003.
- [10] M. Panteli et al., "Learning a Feature Space for Similarity in World Music," in *Proc. ISMIR*, 2015.
- [11] D. Bogdanov et al., "Unifying Low-Level and High-Level Music Similarity Measures," *IEEE Trans. Multimedia*, 2011.
- [12] A. Bonlarron and J. C. Régis, "Intertwining CP and NLP," in *Proc. IJCAI*, 2024.
- [13] E. Palumbo et al., "Text2Tracks: Prompt-Based Music Recommendation," *arXiv*, 2025.
- [14] W. Peng and T. Li, "Music Clustering with Constraints," in *Proc. ICMLA*, 2007.

6. GITHUB

<https://github.com/vanshvekariya/Music-Maven/tree/main/group2>